ESPECIFICAÇÃO E IMPLEMENTAÇÃO DE SERVIÇOS

Integrantes: Edson Pedreira de Oliveira Júnior; Henrique Bassan Rebechi; Matheo Campanelli de Aquino Esteves

Projeto: OrientaTCC – Grupo 11 **Data:** 29/10/2026

1. Bounded Context selecionado

Nome do Bounded Context: Propostas (subdomínio Propostas e aceite da orientação – Core).

Responsabilidade principal no domínio: formalizar o ingresso do aluno na orientação, da submissão da proposta (tema, resumo, objetivos e área de pesquisa) ao aceite do orientador escolhido. O contexto garante uma única proposta ativa por aluno, aplica o limite de orientandos antes do aceite, exige justificativa na devolução e na recusa e publica **PropostaAceita**, que dá origem ao Trabalho de TCC no contexto Acompanhamento do TCC.

Dentre os oito Bounded Contexts do Lab 6, Propostas foi escolhido por ser Core, ter regras de negócio objetivas e verificáveis e depender apenas do Cadastro Acadêmico (IAluno e IOrientador) e da ACL de Identidade e Acesso, o que permite um serviço pequeno, com contrato claro e testável de forma isolada.

2. Capacidades de negócio e serviços candidatos

Liste as capacidades relevantes do Bounded Context. Para cada capacidade, responda às perguntas de análise e indique o(s) serviço(s) candidato(s) proposto(s), se houver.

Capacidade de negócio	Responsabilidade: o que realiza?	Consumidores: quem precisa dela?	Por que disponibilizar por serviço?	Serviço(s) candidato(s) e justificativa
Submeter proposta (submeterProposta)	Registra a proposta do aluno (tema, resumo, objetivos e área de pesquisa) para um orientador, na situação Aguardando orientador; impede uma segunda proposta ativa ou proposta de aluno com Trabalho de TCC em andamento.	Aluno (aplicativo OrientaTCC).	É o comando de entrada do contexto; a regra de proposta única precisa ser aplicada em um único ponto, independentemente do canal de acesso.	**Serviço de Propostas de Orientação.** Opera sobre o agregado Proposta e sua regra de unicidade.
Consultar e listar propostas (consultarProposta, listarPropostas)	Retorna uma proposta ao aluno autor ou ao orientador destinatário e lista propostas por perfil e situação.	Aluno (acompanhar a situação); orientador (propostas recebidas).	Os dois perfis leem o mesmo estado com as mesmas regras de acesso; a interface única evita consultas diretas ao banco.	**Serviço de Propostas de Orientação.** Consultas sobre o mesmo agregado; um serviço separado duplicaria o modelo sem ganho.
Verificar limite de orientandos (verificarLimiteOrientandos)	Calcula orientandos ativos, limite e vagas do orientador a partir do Vínculo de orientação (Núcleo Compartilhado).	Responder proposta (uso interno); aluno e orientador, para saber se há vagas.	A regra decide o aceite e também orienta a escolha do orientador; expô-la evita que o aplicativo recalcule o limite.	**Serviço de Propostas de Orientação.** Não justifica serviço próprio: só o aceite a usa para decidir, na mesma transação.

Capacidade de negócio	Responsabilidade: o que realiza?	Consumidores: quem precisa dela?	Por que disponibilizar por serviço?	Serviço(s) candidato(s) e justificativa
Responder proposta (responderProposta)	Aceita, devolve para ajustes ou recusa a proposta; no aceite, cria o Trabalho de TCC na mesma transação e publica PropostaAceita.	Orientador; Acompanhamento do TCC (consome PropostaAceita).	Concentra as decisões centrais do contexto (situação, limite, justificativa), que precisam ser atômicas e rastreáveis.	**Serviço de Propostas de Orientação.** Operação principal; separar o aceite exigiria transação distribuída com a verificação do limite.
Consultar orientadores e áreas de pesquisa (IOrientador.listarOrientadores)	Lista orientadores com área de pesquisa para o aluno escolher a quem submeter a proposta.	Aluno ao montar a proposta; Reuniões de Orientação.	Já é disponibilizada como Open Host Service pelo Cadastro Acadêmico (Lab 6).	**Nenhum serviço novo em Propostas.** Permanece no Cadastro Acadêmico; Propostas apenas consome IOrientador.

Justificativa da decomposição: explique por que as responsabilidades foram agrupadas em um serviço ou separadas em serviços distintos. Considere coesão, consumidores e fronteiras do domínio.

Coesão. As capacidades 1 a 4 foram agrupadas no Serviço de Propostas de Orientação porque operam sobre o mesmo agregado (Proposta) e o mesmo ciclo de vida (Aguardando orientador → Aceita, Devolvida ou Recusada). As regras de proposta única por aluno, limite de orientandos e justificativa obrigatória dependem desse estado e precisam ser verificadas de forma consistente; dividi-las exigiria chamadas remotas e transação distribuída para decidir um único aceite.

Consumidores. Os consumidores são os mesmos (aluno e orientador, pelo aplicativo, e o Acompanhamento do TCC, que reage a PropostaAceita). Uma única interface atende a todos, sem integrações específicas por cliente.

Fronteiras. A consulta de alunos e orientadores (capacidade 5) pertence ao Cadastro Acadêmico, que já a fornece como OHS; a autenticação é de Identidade e Acesso (Ator traduzido pela ACL); a gestão do trabalho após o aceite é do Acompanhamento do TCC. O serviço não altera dados desses contextos: consome IAluno/IOrientador e publica eventos. A verificação do limite usa apenas o modelo mínimo do Vínculo de orientação do Núcleo Compartilhado, por isso fica no mesmo serviço do aceite.

3. Especificação do serviço selecionado

Nome do serviço: Serviço de Propostas de Orientação (propostas-service).

Capacidade(s) de negócio que disponibiliza: submeterProposta, consultarProposta, listarPropostas, verificarLimiteOrientandos e responderProposta.

Responsabilidade do serviço: gerenciar o ciclo de vida da proposta de orientação (submissão, consulta e resposta do orientador), aplicando a proposta única por aluno, o limite de orientandos e a justificativa obrigatória, e iniciar o vínculo de orientação no aceite (criação do Trabalho de TCC e publicação de PropostaAceita).

Consumidores previstos: aplicativo OrientaTCC (telas do aluno e do orientador) e o contexto Acompanhamento do TCC (evento PropostaAceita e Núcleo Compartilhado). PropostaDevolvida e PropostaRecusada são publicados sem assinantes no recorte atual, como definido no Lab 6.

Forma de comunicação prevista: API REST síncrona (HTTP/JSON), documentada em OpenAPI 3, para comandos e consultas; eventos de domínio assíncronos (PropostaAceita, PropostaDevolvida, PropostaRecusada) publicados após a confirmação da transação, no envelope da Linguagem Publicada (evento, versao, ocorridoEm, trabalhoId, dados). O serviço consome IAluno/IOrientador do Cadastro Acadêmico e recebe o Ator {usuarioId, perfil} da ACL de Identidade e Acesso.

Justificativa da escolha: o serviço concentra as regras mais objetivas de um contexto Core (proposta única, limite de orientandos, justificativa obrigatória e aceite atômico) e é a porta de entrada do fluxo de orientação: todo Trabalho de TCC nasce de uma proposta aceita. Por ter poucas dependências, pode ser implementado e testado de forma isolada.

3.1 Interface e operações

Operação	Finalidade	Entradas principais	Resultado esperado
submeterProposta `POST /propostas`	Registrar a proposta do aluno para um orientador.	Ator aluno; orientadorId, tema, resumo, objetivos, areaPesquisa.	201 com a proposta em aguardando_orientador. Erros: 409 (proposta ativa ou trabalho em andamento), 404 (orientador inexistente), 422 (dados inválidos).
consultarProposta `GET /propostas/{propostaId}`	Ver os dados e a situação de uma proposta.	Ator; propostaId.	200 com a proposta. Erros: 403 (ator não é o aluno autor nem o orientador destinatário), 404.
listarPropostas `GET /propostas?situacao=`	Listar as propostas do aluno ou as recebidas pelo orientador.	Ator; situacao (opcional).	200 com a lista ordenada por data de submissão, filtrada pelo perfil do ator.
verificarLimiteOrientandos `GET /orientadores/{orientadorId}/limite-orientandos`	Informar se o orientador ainda pode aceitar orientandos.	orientadorId.	200 com limite, orientandosAtivos, vagasDisponiveis e podeAceitar. Erro: 404.
responderProposta `POST /propostas/{propostaId}/resposta`	Aceitar, devolver para ajustes ou recusar a proposta.	Ator orientador; propostaId; decisao; justificativa.	200 com a proposta respondida, o Trabalho de TCC (no aceite) e o evento publicado. Erros: 401, 403, 404, 409 e 422 (ver 3.2).

3.2 Contrato de uma operação

Operação escolhida para detalhamento: responderProposta – `POST /propostas/{propostaId}/resposta`

Elemento	Especificação
Parâmetros de entrada	**Caminho:** propostaId (texto, obrigatório). **Cabeçalhos (Ator traduzido pela ACL):** X-Usuario-Id e X-Perfil (aluno | orientador). **Corpo JSON:** decisao (obrigatório: aceitar | devolver | recusar); justificativa (texto, até 2000 caracteres; obrigatória para devolver e recusar). Campos não previstos são rejeitados. Exemplo: `{"decisao": "devolver", "justificativa": "Delimitar melhor o escopo e os objetivos."}`
Pré-condições	**PC1.** Ator autenticado com perfil orientador. **PC2.** A proposta existe. **PC3.** O ator é o orientador destinatário da proposta. **PC4.** A proposta está em aguardando_orientador (ainda não foi respondida).
Regras de negócio	**RN1.** No aceite, o aluno não pode ter Trabalho de TCC em andamento. **RN2.** No aceite, os orientandos ativos do orientador devem ser menos que o seu limite de orientandos (verificarLimiteOrientandos). **RN3.** Devolver e recusar exigem justificativa com no mínimo 10 caracteres, desconsiderados os espaços nas extremidades; no aceite, ela é opcional. **RN4.** Aceitar → aceita, com criação do Trabalho de TCC (em_orientacao) e do Vínculo de orientação na mesma transação; devolver → devolvida; recusar → recusada. Uma proposta respondida não recebe nova resposta (o aluno pode submeter nova proposta). **RN5.** Após a confirmação, publica PropostaAceita, PropostaDevolvida ou PropostaRecusada. Em caso de erro, nada é alterado e nenhum evento é publicado.

Elemento	Especificação
Resultado / saída	**HTTP 200** com: • proposta: id, situacao, decisao, justificativa, respondidaEm e trabalhoId; • trabalho: id, propostaId, alunoId, orientadorId, situacao (em_orientacao) e iniciadoEm; null quando não há aceite; • eventoPublicado: evento, versao, ocorridoEm, trabalhoId e dados.
Erros ou condições de falha	Formato: `{"erro": {"codigo", "mensagem", "detalhes"}}`. **401** NAO_AUTENTICADO / PERFIL_INVALIDO: cabeçalhos ausentes ou perfil desconhecido. **403** PERFIL_NAO_PERMITIDO (PC1); ORIENTADOR_NAO_DESTINATARIO (PC3). **404** PROPOSTA_NAO_ENCONTRADA (PC2). **409** PROPOSTA_JA_RESPONDIDA (PC4); TRABALHO_EM_ANDAMENTO (RN1); LIMITE_ORIENTANDOS_ATINGIDO (RN2). **422** JUSTIFICATIVA_OBRIGATORIA (RN3); DADOS_INVALIDOS: decisão fora do domínio, tipo incorreto, campo extra ou justificativa acima de 2000 caracteres.

4. Implementação

Tecnologias utilizadas: Python 3.13, FastAPI 0.162 (API REST e OpenAPI 3 gerado automaticamente), Pydantic 2 (DTOs e validação de entrada), Uvicorn (servidor ASGI), pytest 9 com TestClient/httpx (testes automatizados), armazenamento em memória e Git. A lógica de domínio não depende do framework e pode ser portada para o módulo de propostas da API do OrientaTCC.

Repositório GitHub: (a preencher após o push)

Localização dos arquivos relevantes (pasta, módulo ou classe): pasta `servico/` (também entregue em servico.zip).

- `app/dominio.py`: entidades Proposta, TrabalhoTCC, Ator e EventoDominio, situações, decisões e erros de domínio.
- `app/servico.py`: classe ServicoPropostas, com os casos de uso; o contrato está em responder_proposta.
- `app/repositorio.py`: repositório em memória, Vínculo de orientação e dados fictícios de demonstração.
- `app/eventos.py`: barramento interno de eventos; `app/esquemas.py`: DTOs da Linguagem Publicada.
- `app/main.py`: rotas REST, ACL do Ator (obter_ator) e tratamento de erros.
- `tests/test_propostas.py`: testes automatizados; `docs/openapi.json`: especificação OpenAPI; `README.md`: execução.

Explique brevemente como a implementação realiza as regras definidas no contrato:

A entrada é validada pelo DTO ResponderPropostaEntrada (Pydantic): decisao é um enum, justificativa tem até 2000 caracteres e campos extras são proibidos; violações geram 422 DADOS_INVALIDOS. A dependência obter_ator traduz os cabeçalhos para o Ator, o que equivale à ACL de Identidade e Acesso (401 quando ausente).

Em ServicoPropostas.responder_proposta, as pré-condições são verificadas em ordem (PC1 perfil → 403, PC2 existência → 404, PC3 destinatário → 403, PC4 situação → 409), seguidas de RN3 (justificativa → 422). No aceite, RN1 e RN2 consultam o Vínculo de orientação (trabalho em andamento e verificar_limite_orientandos) e geram 409.

Todo o trecho roda dentro de `repo.transacao` (lock), de modo que verificar o limite, criar o Trabalho de TCC e mudar a situação da proposta formam uma unidade atômica (RN4). O evento só é publicado no barramento depois da transação (RN5). Cada erro de domínio é uma exceção com status HTTP e código próprio, convertida por um exception handler no formato `{"erro": {...}}`.

A implementação está acessível por meio de uma interface (por exemplo, endpoint/API ou interface equivalente)? Sim. A operação é exposta pelo endpoint REST `POST /propostas/{propostaId}/resposta` (além

das demais operações da Seção 3.1). Ao executar `uvicorn app.main:app`, a API fica em http://127.0.0.1:8000, com documentação interativa (Swagger UI) em `/docs` e especificação em `/openapi.json`, também exportada em `docs/openapi.json`.

5. Testes e evidências

Registre testes que demonstrem o comportamento esperado e o tratamento de pelo menos uma condição de erro.

Cenário de teste	Dados / condição	Resultado esperado	Resultado obtido / evidência
CT01 – Aceite válido	prop-0001 (aluno-02 → prof-01); prof-01 com 1 de 3 orientandos; decisao = aceitar.	200; proposta aceita; Trabalho de TCC em_orientacao; PropostaAceita publicado; orientandos ativos de 1 para 2.	Aprovado (test_ct01, Fig. 1). curl nº 5: HTTP 200, tcc-0004 criado e PropostaAceita (Fig. 3).
CT02 – Devolver com justificativa	prop-0001; decisao = devolver; justificativa de 41 caracteres.	200; proposta devolvida; trabalho null; PropostaDevolvida publicado.	Aprovado (test_ct02, Fig. 1).
CT03 – Recusar sem justificativa (RN3)	prop-0001; decisao = recusar, sem justificativa.	422 JUSTIFICATIVA_ OBRIGATORIA; proposta continua aguardando_orientador; nenhum evento.	Aprovado (test_ct03). curl nº 3: HTTP 422 (Fig. 2).
CT04 – Limite de orientandos atingido (RN2)	prop-0002 (aluno-03 → prof-02); prof-02 com 2 de 2 orientandos; decisao = aceitar.	409 LIMITE_ORIENTANDOS_ ATINGIDO; nenhum trabalho criado; nenhum evento.	Aprovado (test_ct04). curl nº 2: HTTP 409, “Limite de orientandos atingido (2/2)” (Fig. 2).
CT05 – Proposta já respondida (PC4)	prop-0001 já aceita; nova resposta recusar.	409 PROPOSTA_JA_ RESPONDIDA.	Aprovado (test_ct05). curl nº 6: HTTP 409 (Fig. 3).
CT06 – Orientador não destinatário (PC3)	prof-02 responde prop-0001.	403 ORIENTADOR_NAO_ DESTINATARIO.	Aprovado (test_ct06, Fig. 1).
CT07 – Aluno tenta responder (PC1)	aluno-02 com perfil aluno.	403 PERFIL_NAO_PERMITIDO.	Aprovado (test_ct07, Fig. 1).
CT08 – Entrada inválida	decisao = "talvez".	422 DADOS_INVALIDOS, campo decisao.	Aprovado (test_ct08). curl nº 4: HTTP 422 (Fig. 2).
CT09 – Proposta inexistente (PC2)	propostaId = prop-9999.	404 PROPOSTA_NAO_ ENCONTRADA.	Aprovado (test_ct09, Fig. 1).
CT10 – Sem autenticação	Requisição sem X-Usuario-Id e X-Perfil.	401 NAO_AUTENTICADO.	Aprovado (test_ct10, Fig. 1).
CT11 – Submeter proposta válida	aluno-01, sem proposta ativa, para prof-01.	201; proposta em aguardando_ orientador.	Aprovado (test_ct11). curl nº 7: HTTP 201, prop-0003 (curl_ demo.log).
CT12 – Segunda proposta ativa	aluno-02 já possui prop-0001 aguardando.	409 PROPOSTA_ATIVA_ EXISTENTE.	Aprovado (test_ct12, Fig. 1).
CT13 – Campos inválidos na submissão	tema = "TCC" (menos de 5 caracteres); resumo vazio.	422 DADOS_INVALIDOS, com os campos tema e resumo.	Aprovado (test_ct13, Fig. 1).

Links ou referências às evidências (testes, capturas de tela, logs ou execução):

Resultado consolidado: em 29/10/2026, `python -m pytest -v` executou 20 testes automatizados: 20 aprovados e 0 falhas (0,29 s). Além dos cenários CT01–CT13, os testes cobrem justificativa só com espaços, aluno com trabalho em andamento, nova submissão após devolução, orientador inexistente, consulta por terceiro, listagem por perfil e situação e verificação do limite.

Repositório GitHub: (a preencher após o push), pasta `docs/evidencias/`. Arquivos entregues junto ao relatório:

- `evidencias/testes_pytest.log` – saída completa dos testes; `01_testes_pytest.png` (Figura 1).
- `evidencias/curl_demo.log` – oito requisições curl com respostas; `02_curl_condicoes_de_erro.png` (Figura 2) e `03_curl_aceite_e_proposta_ja_respondida.png` (Figura 3).
- `evidencias/servidor_uvicorn.log` – log do servidor com os status HTTP (200, 409, 422, 422, 200, 409, 201, 200); `evidencias/git_log.txt` – histórico de commits.

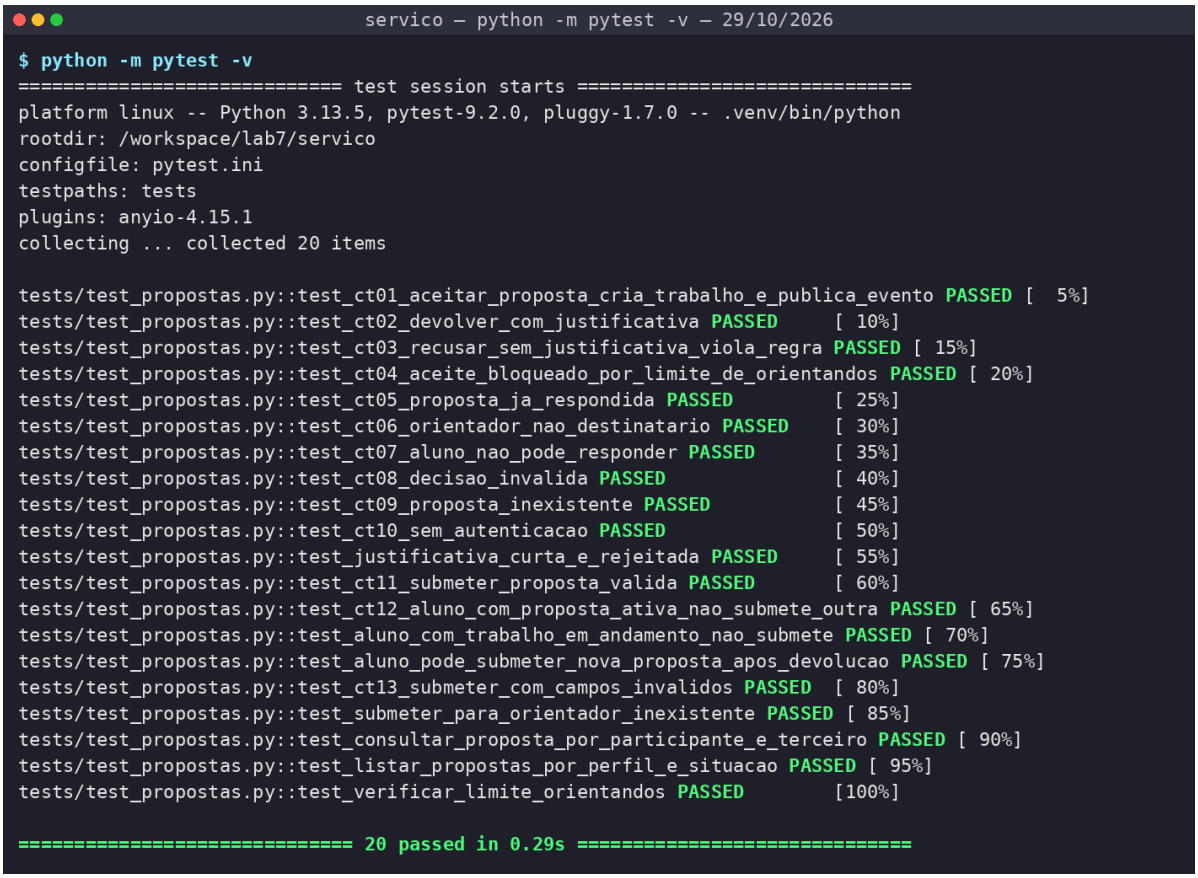


Figura 1 – Execução dos testes automatizados: 20 aprovados (pytest, 29/10/2026).

curl — responderProposta: condições de erro (409 e 422)

```
### 2. responderProposta — aceite bloqueado pelo limite de orientandos
$ curl -s -X POST http://127.0.0.1:8000/propostas/prop-0002/resposta -H 'X-Usuario-Id: prof-02' -H
  'X-Perfil: orientador' -d '{"decisao":"aceitar"}'
{
  "erro": {
    "codigo": "LIMITE_ORIENTANDOS_ATINGIDO",
    "mensagem": "Limite de orientandos atingido (2/2)."
  }
}
HTTP 409

### 3. responderProposta — recusa sem justificativa (regra de negócio)
$ curl -s -X POST http://127.0.0.1:8000/propostas/prop-0001/resposta -H 'X-Usuario-Id: prof-01' -H
  'X-Perfil: orientador' -d '{"decisao":"recusar"}'
{
  "erro": {
    "codigo": "JUSTIFICATIVA_OBRIGATORIA",
    "mensagem": "A decisão 'recusar' exige justificativa com pelo menos 10 caracteres."
  }
}
HTTP 422

### 4. responderProposta — entrada inválida
$ curl -s -X POST http://127.0.0.1:8000/propostas/prop-0001/resposta -H 'X-Usuario-Id: prof-01' -H
  'X-Perfil: orientador' -d '{"decisao":"talvez"}'
{
  "erro": {
    "codigo": "DADOS_INVALIDOS",
    "mensagem": "Os dados de entrada são inválidos.",
    "detalhes": [
      {
        "campo": "decisao",
        "problema": "valor inválido; use 'aceitar', 'devolver' ou 'recusar'"
      }
    ]
  }
}
HTTP 422
```

Figura 2 – responderProposta via curl: limite de orientandos (409), recusa sem justificativa (422) e entrada inválida (422).



Figura 3 – responderProposta via curl: aceite com criação do Trabalho de TCC e evento PropostaAceita (200) e nova resposta à proposta já aceita (409).